



HEI-Vs Engineering School - Industrial Automation Base

Cours AutB

Author: [Cédric Lenoir](#)

Version 2026, Version 1.0

LAB 02 Programmation structurée d'un convoyeur

Préambule

Pour des raisons historiques, nous utilisons le PLC dans le ctrlX Core pour la programmation et le PLC Siemens S7 pour la communication avec les entrées et les sorties. Ce n'est pas forcément très pratique, mais c'est aussi conforme à la réalité de l'industrie. Il est rare d'avoir la chance de pouvoir concevoir une nouvelle installation, ou une nouvelle machine en faisant abstraction du passé.

Le PLC Siemens écrit ses entrées sur des variables globales du ctrlX et lit les variables globales de sortie pour les envoyer vers les actionneurs.

Toutes les entrées et sorties sont disponibles dans le ctrlX Core via la structure globale :

`GVL_Abox.uaAboxInterface` .

Objectifs

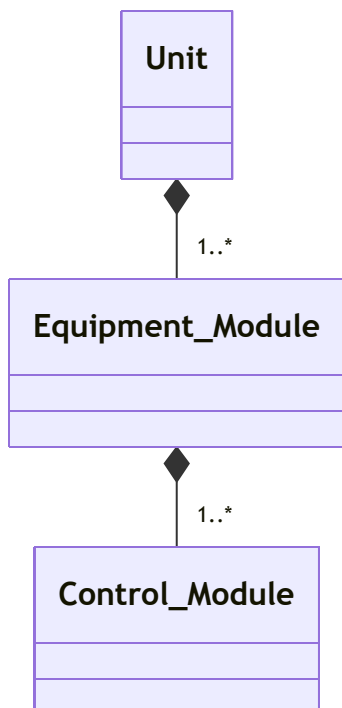
- Programmer une structure de données pour interfacer les entrées/sorties d'un convoyeur.
- Programmer plusieurs POU (Program Organization Unit) de manière structurée pour piloter un convoyeur.

Il existe des standards qui définissent des structures pour les machines dans l'industrie. L'intérêt de ces structures est de faciliter la réutilisabilité du code sur plusieurs projets. Nous nous basons ici sur le standard ISA 88.

- Le convoyeur est défini comme un **Equipment Module**.
- Le convoyeur est composé de plusieurs **Control Modules**.
- Le **Control Module** est le plus petit module défini dans la norme **ISA 88**.
- Un **Equipment Module** est constitué de plusieurs **Control Modules**.
- Une machine, **Unit** est constituée de plusieurs **Equipment Modules**.

Nous considérons ici notre convoyeur comme un élément d'une machine, **Unit**.

La machine peut elle-même faire partie d'un ensemble de machines.

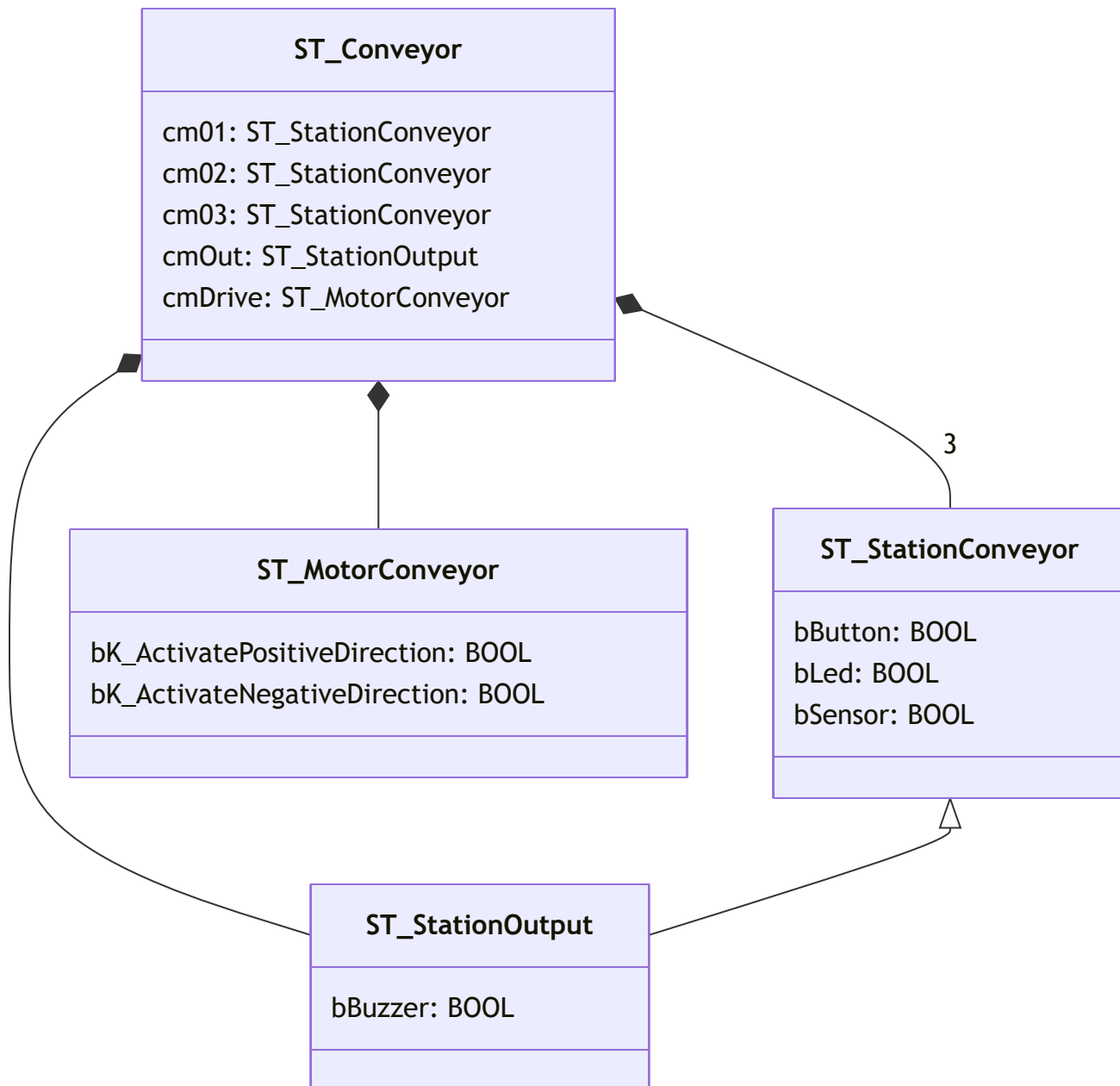


Unit based on S88

1ère étape

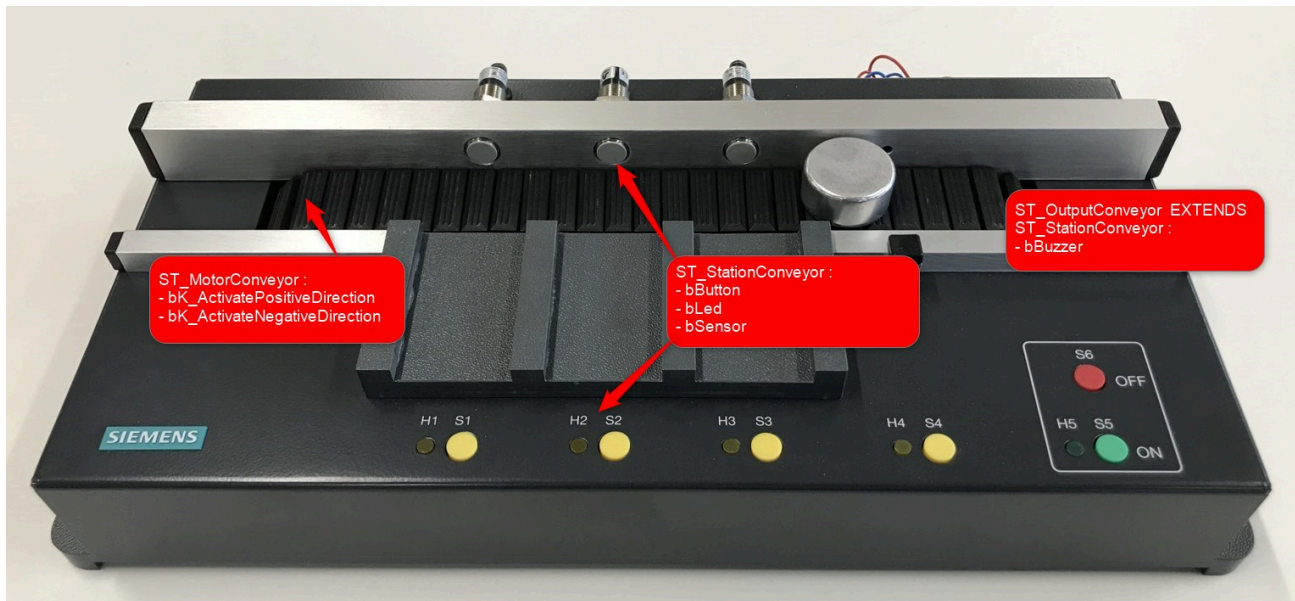
Objectif :

Programmer des **DUT** (Data User Type) sous forme de structures de données qui permettront de regrouper plusieurs variables (qui peuvent être de différents types de données) dans une unité logique cohérente.



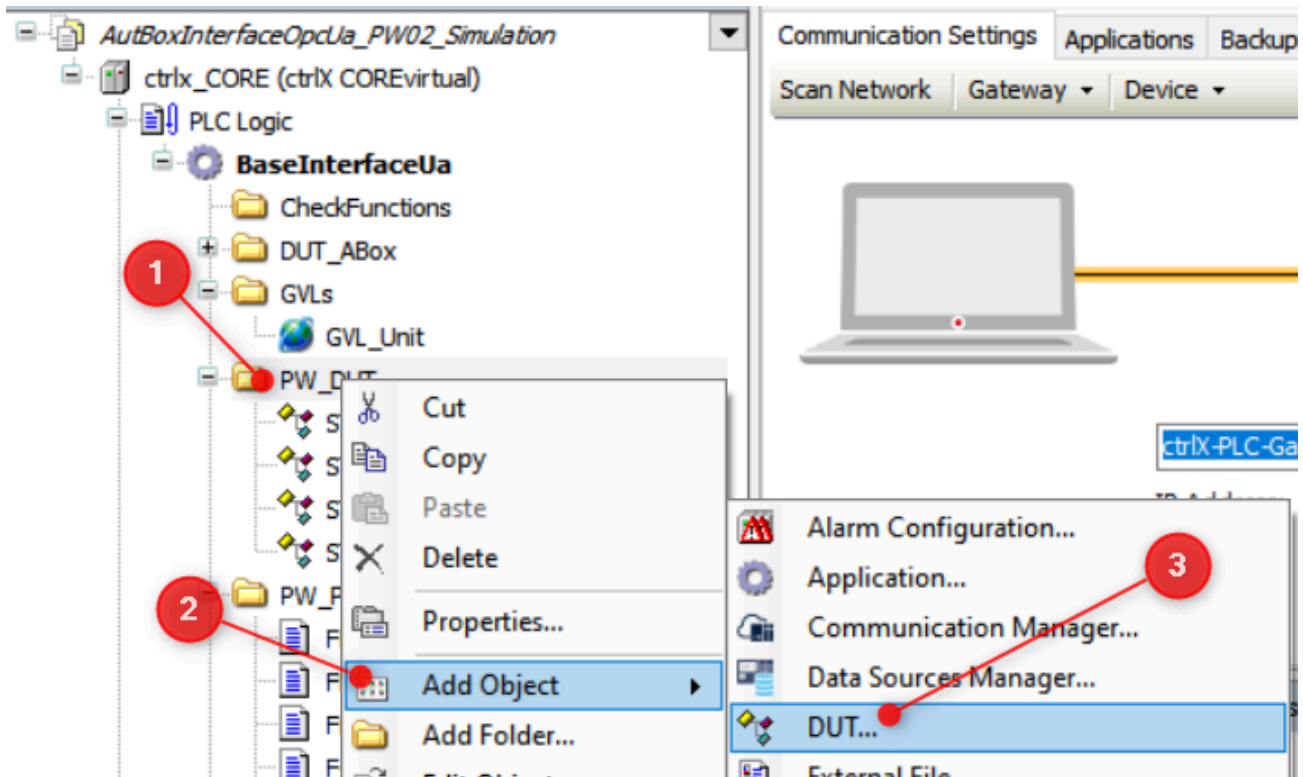
Structures de données du convoyeur

Remarque : ST_StationOutput "hérite" de ST_StationConveyor.



Structures de données du convoyeur.

Comment ajouter un DUT (Data User Type) ?



Ajouter un DUT (Data User Type)

2ème étape

Objectifs :

- Créer une instance de la structure de données `ST_Conveyor` dans le programme `PLC_PRG` et compléter le programme afin de lier les entrées et les sorties du convoyeur avec cette instance.
- Vérifier le fonctionnement des entrées /sorties du convoyeur.

Remarque : La structure de données `GVL_Abox.uaAboxInterface` représente l'interface avec les entrées/sorties du PLC.

Compléter votre programme principal `PLC_PRG` en vous référant à l'exemple ci-dessous.

```
PROGRAM PLC_PRG
VAR
    diMyLoop          : DINT;
    emConveyor        : ST_Conveyor; // Your structure
END_VAR
//
// Manage inputs
//
// cm01
emConveyor.cm01.bButton := GVL_Abox.uaAboxInterface.uaDigitalIn.Input_0_0;
// other inputs

//
// Code
//

//
// Manage outputs
//
// cm01
GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_0 := emConveyor.cm01.bLed;
// Other outputs
```

PLC Tags, SDS, Software Design Specification

Siemens Address	Data Type	ctrlX Global Var Struct	Hardware on conveyor
%I0.0	BOOL	GVL_Abox.uaAboxInterface.uaDigitalIn.Input_0_0	Push Button S1
%I0.1	BOOL	GVL_Abox.uaAboxInterface.uaDigitalIn.Input_0_1	Push Button S2
%I0.2	BOOL	GVL_Abox.uaAboxInterface.uaDigitalIn.Input_0_2	Push Button S3
%I0.3	BOOL	GVL_Abox.uaAboxInterface.uaDigitalIn.Input_0_3	Push Button S4
%I0.4	BOOL	GVL_Abox.uaAboxInterface.uaDigitalIn.Input_0_4	Sensor Active B1
%I0.5	BOOL	GVL_Abox.uaAboxInterface.uaDigitalIn.Input_0_5	Sensor Active B2
%I0.6	BOOL	GVL_Abox.uaAboxInterface.uaDigitalIn.Input_0_6	Sensor Active B3
%I0.7	BOOL	GVL_Abox.uaAboxInterface.uaDigitalIn.Input_0_7	Sensor Not Active B4
%Q0.0	BOOL	GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_0	Led H1
%Q0.1	BOOL	GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_1	Led H2
%Q0.2	BOOL	GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_2	Led H3
%Q0.3	BOOL	GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_3	Led H4
%Q0.4	BOOL	GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_4	Contactor K1
%Q0.5	BOOL	GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_5	Contactor K2
%Q0.6	BOOL	GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_6	Buzzer A1

Visualisation des données du programme

Le traitement d'un programme sur un automate est cyclique. **L'analyse step-by-step n'est en général pas possible.**

La facilité d'accès aux données pendant le fonctionnement du programme est primordiale.

L'accès aux données doit être prise en compte dès la phase de conception du programme.

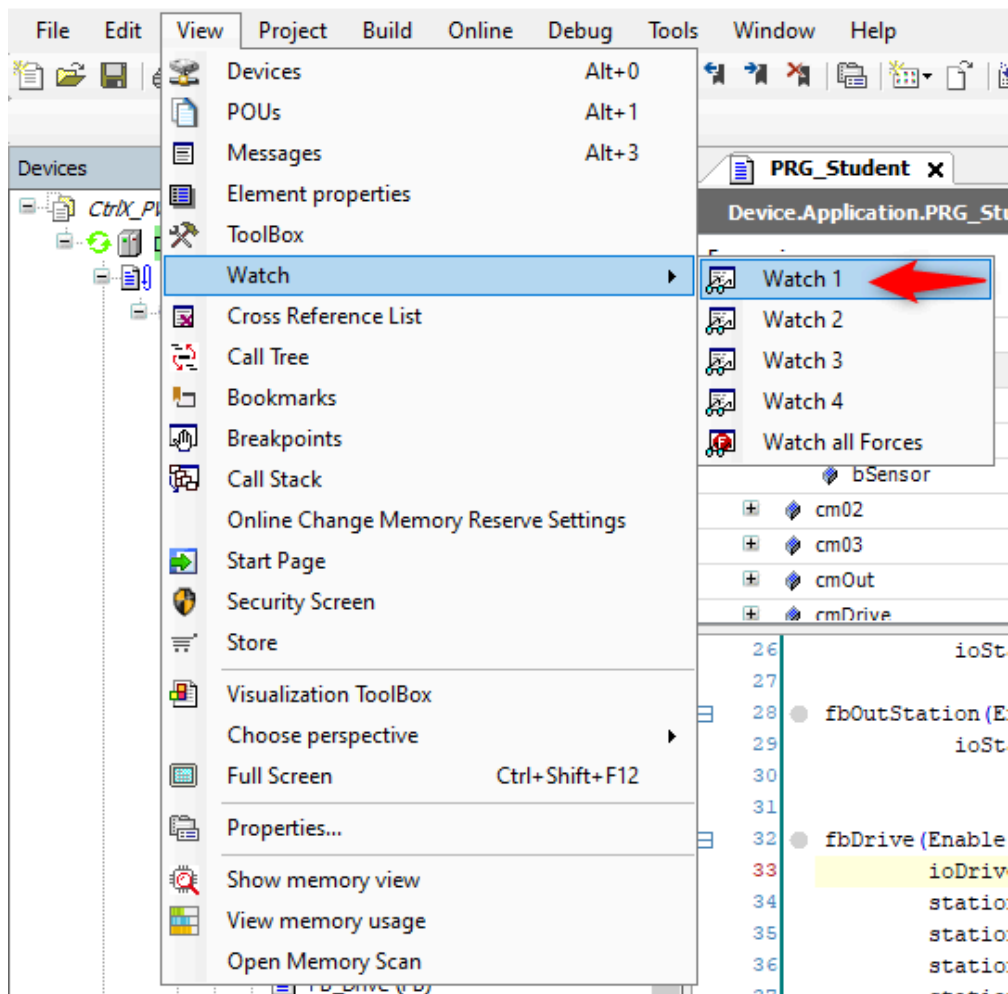
Commande en mode manuel du convoyeur avec l'outil "Watch"

Il existe dans chaque environnement de développement des PLC (IDE : Integrated Development Environment) la possibilité de visualiser et de forcer des variables.

Vérifier le fonctionnement des entrées/sorties du convoyeur à l'aide de l'outil "Watch".

Marche à suivre :

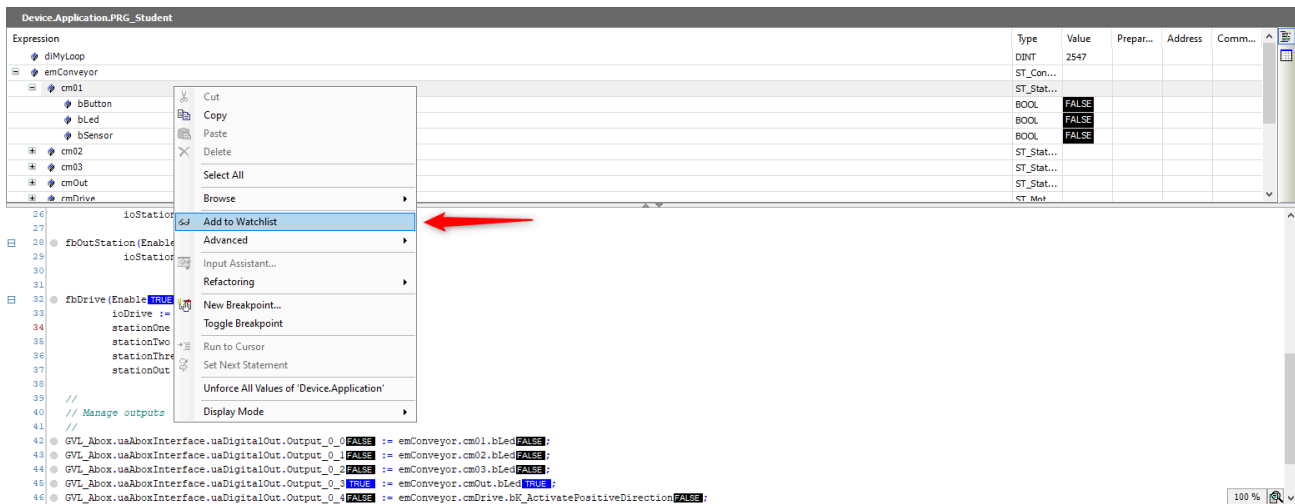
1. Ouvrir une fenêtre "Watch"



Ajouter une fenêtre "Watch"

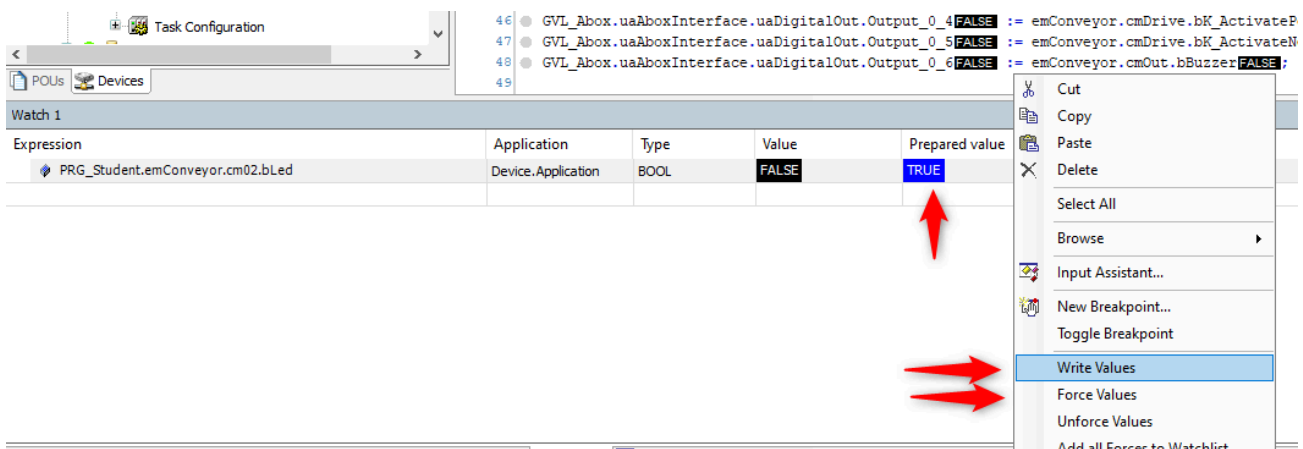
2. Sélectionner l'élément désiré

Il est possible de sélectionner tout élément qui a été instancié, à savoir une variable, une structure ou un bloc fonctionnel.



Ajouter une élément dans une fenêtre "Watch"

3. Visualiser/forcer un élément



Visualiser/forcer un élément dans une fenêtre "Watch"

Raccourcis :

Pour forcer --> [F7]

Pour écrire --> [Ctrl + F7]

Remarque :

Forcer une variable signifie que vous imposerez une valeur spécifique à une variable, indépendamment de ce que le programme pourrait essayer de lui assigner. Une fois forcée, la variable conserve cette valeur jusqu'à ce que vous la libériez du forçage. Ceci est utile pour les tests et le débogage.

Écrire une variable signifie simplement attribuer une valeur à une variable dans le cours normal de l'exécution du programme. Cette valeur peut être modifiée à mesure que le programme s'exécute et

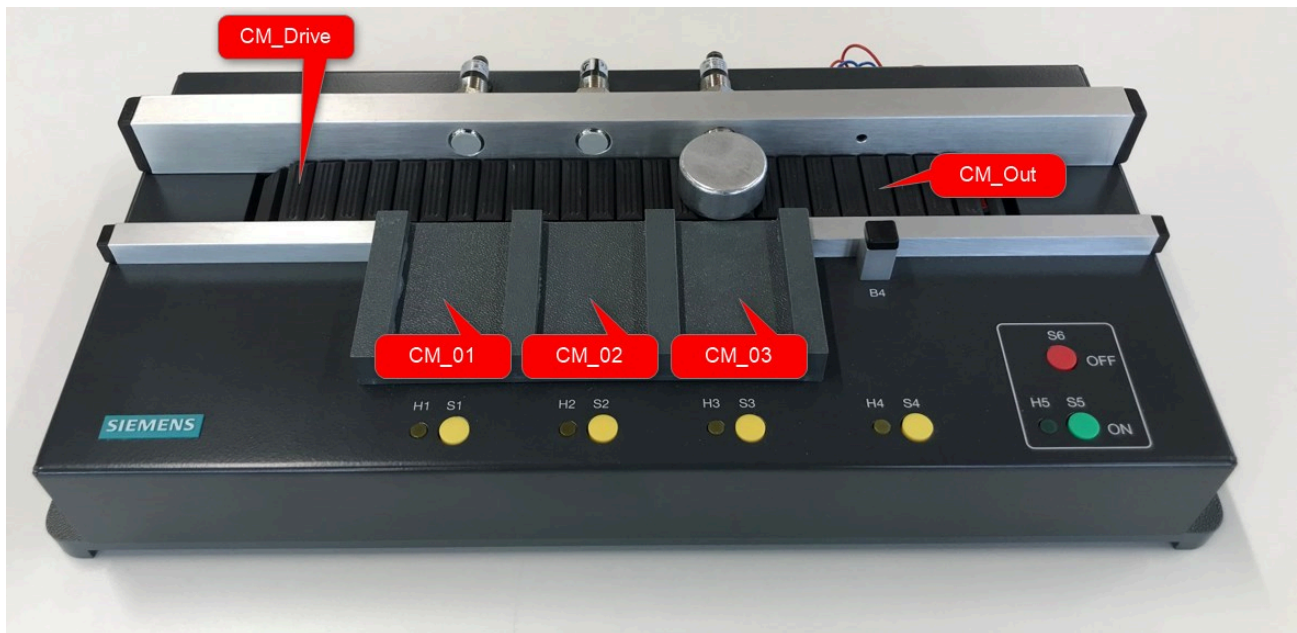
suit les règles et la logique définies dans le code.

3ème étape

POU (Program Organization Unit)

Objectifs :

- Programmer des **FB (Function Block)** qui vont permettre de piloter différents **Control Module (CM)**.
- Compléter le **programme** principal PLC_PRG (considéré ici comme un **Equipment Module (EM)**).



Structure du convoyeur selon ISA 88

- Le convoyeur est constitué de trois **Control Modules** identiques dédiés aux stations intermédiaires (--> CM_01, CM_02, CM_03).
- La station de sortie est constituée d'un **Control Module** dédié à la commande du moteur et d'un buzzer (--> CM_Out).

Remarque : Le capteur optique B4 est N.C. (Normally Closed) contrairement aux autres capteurs inductifs B1, B2 et B3.

- Le convoyeur est constitué d'un **Control Module** dédié à la commande du moteur (--> CM_Drive).

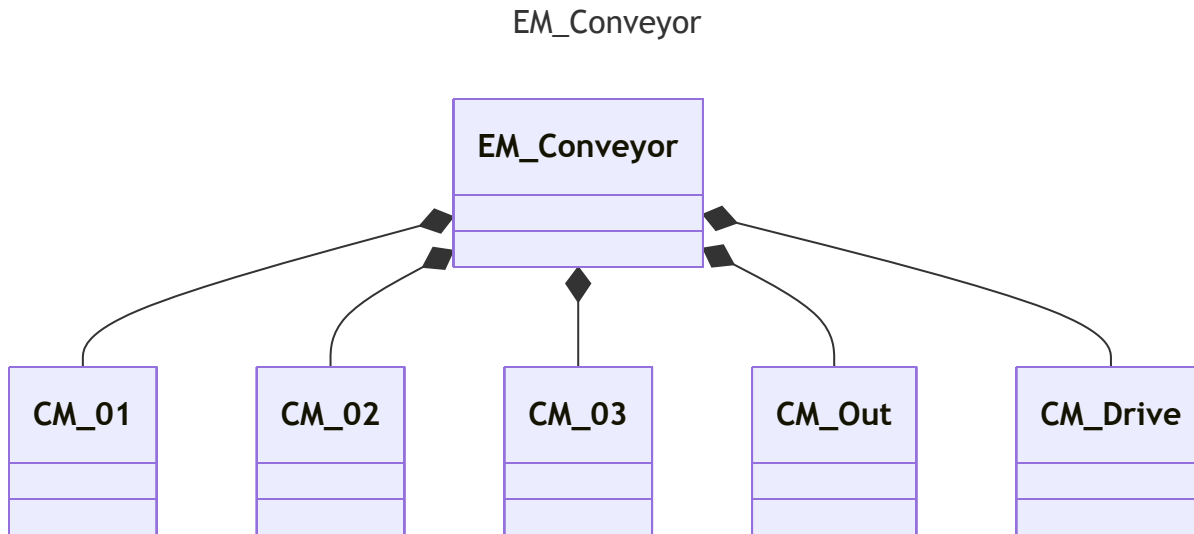
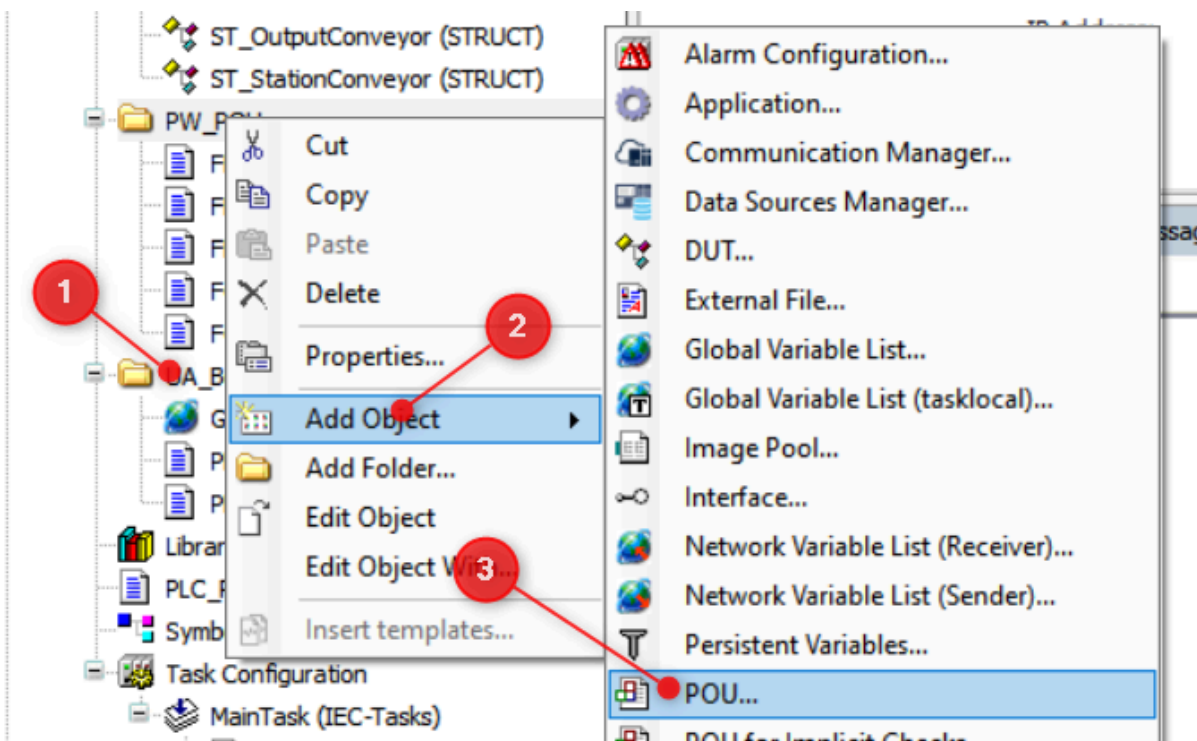


Diagramme d'objet du convoyeur

Comment ajouter un POU ?



Ajouter un POU (Program Organization Unit)

Description générale du fonctionnement

1. Le convoyeur avance si aucune pièce n'est présente.
2. Si l'on pose une pièce à l'entrée, elle avance jusqu'à la première station, puis s'arrête et incrémente le compteur de la station.
3. Si l'on appuie sur le bouton de la station, le convoyeur redémarre et amène la pièce à la station suivante.
4. L'arrivée de la pièce sur la dernière station déclenche une alarme sonore.

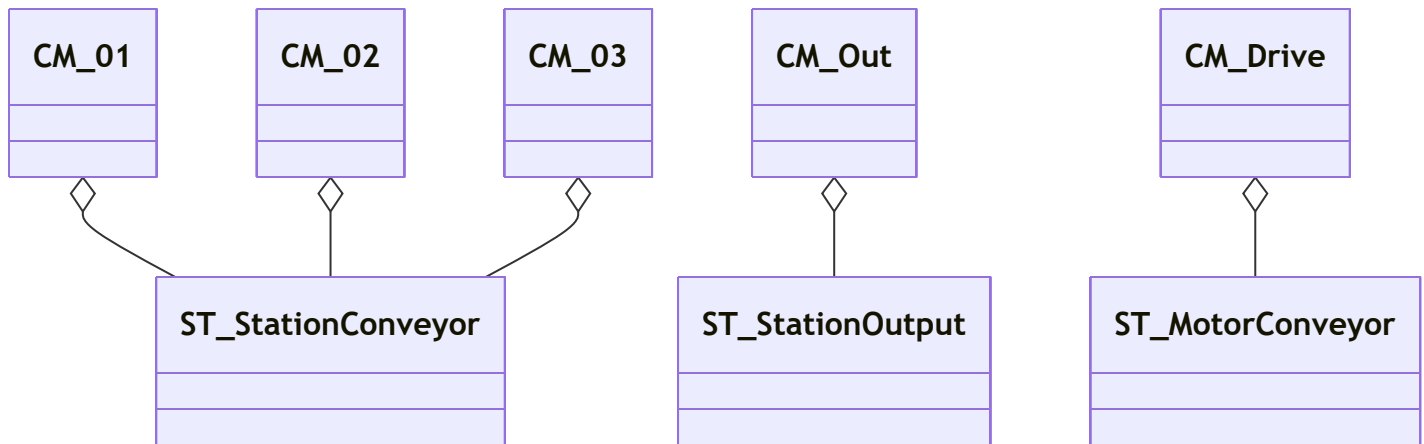
Programmer les différents **Control Modules** en respectant les URS (User Request Specification) ci-dessous.

1) FB_Station

URS (User Request Specification)

- Si l'entrée `Enable` est `FALSE`, le **FB** est inactif.
- Une variable `VAR_IN_OUT` permet de passer la structure `ST_StationConveyor` en paramètre.
- une variable de sortie `diCounter` compte le nombre de pièces qui sont détectées par le capteur en entrée de station.
- au moment où une pièce est détectée par le capteur, une sortie `stop` est activée à `TRUE`, une sortie `release` est mise à `FALSE`.
- La sortie hardware `bLed` prend la valeur de `stop`.
- Si l'on appuie sur le bouton `bButton`, la sortie `release` passe à `TRUE` et la sortie `stop` passe à `FALSE`.
- Si l'on appuie pendant deux secondes ou plus sur le bouton de la station, la variable `diCounter` passe à ``0``.

Principe VAR_IN_OUT



2) FB_OutStation

URS (User Request Specification)

- La structure de donnée VAR_IN_OUT est ST_OutputConveyor
- Ce FB a absolument la même manière que FB_Station à une exception près: la variable stop active le Buzzer .

3) FB_Drive

URS (User Request Specification)

- Si l'entrée Enable est FALSE , le **FB** est inactif.
- Ce FB reçoit les 4 stations en paramètre, VAR_IN_OUT , ainsi que la structure hardware ST_MotorConveyor .
- Si l'une des variables stop des 4 stations est TRUE , le convoyeur s'arrête. Sinon il avance dans la direction S1 --> S4.

Appel des "Control Modules" depuis le programme principal

Compléter le programme principal PLC_PRG en intégrant l'appel des Control Modules en vous référant à l'exemple ci-dessous:

```
PROGRAM PLC_PRG
VAR
    // If not in test mode, Function Blocks below are enabled
    testMode          : BOOL;

    fbStationOne      : FB_Station;
    fbStationTwo       : FB_Station;
    fbStationThree     : FB_Station;
    fbOutStation       : FB_OutStation;
    fbDrive            : FB_Drive;
END_VAR
```

```

diMyLoop := diMyLoop + 1;

// Manage inputs
// cm01
emConveyor.cm01.bButton := GVL_Abox.uaAboxInterface.uaDigitalIn.Input_0_0;
...
// cm02
emConveyor.cm02.bButton := GVL_Abox.uaAboxInterface.uaDigitalIn.Input_0_1;
...
...

// Execute Control Modules
fbStationOne(Enable := NOT testMode,
             ioStation := emConveyor.cm01);
fbStationTwo(Enable := NOT testMode,
             ioStation := emConveyor.cm02);
fbStationThree(Enable := NOT testMode,
              ioStation := emConveyor.cm03);

fbOutStation(Enable := NOT testMode,
            ioStation := emConveyor.cmOut);

fbDrive(Enable := NOT testMode,
        ioDrive := emConveyor.cmDrive,
        stationOne := fbStationOne,
        stationTwo := fbStationTwo,
        stationThree := fbStationThree,
        stationOut := fbOutStation);

// Manage outputs
//
GVL_Abox.uaAboxInterface.uaDigitalOut.Output_0_0 := emConveyor.cm01.bLed;
...
...

```

Mode "Test"

La variable `testMode` de type `BOOL` peut être ajoutée pour permettre de désactiver l'appel des Control Modules.

```
PROGRAM PLC_PRG
VAR
    // If not in test mode, Function Blocks below are enabled
    testMode          : BOOL;

    ...
END_VAR

//
// Execute Control Modules
fbStationOne(Enable := NOT testMode,
             ioStation := emConveyor.cm01);

...
```

Finalisation du programme

En vous référant au programme `PLC_PRG` ci-dessous, ajouter :

- une fonction qui gèrent les entrées (--> `FC_GetInput`)
- une fonction qui gèrent les sorties (--> `FC_SetOutput`)

```
PROGRAM PLC_PRG
VAR
    // If not in test mode, Function Blocks below are enabled
    testMode          : BOOL;

    fbStationOne       : FB_Station;
    fbStationTwo       : FB_Station;
    fbStationThree     : FB_Station;
    fbOutStation       : FB_OutStation;
    fbDrive            : FB_Drive;

END_VAR
```



```
diMyLoop := diMyLoop + 1;

// Manage inputs
FC_GetInput(ioHwInterface := GVL_Abox.uaAboxInterface,
            ioPhysicalControl := emConveyor);

// Execute Control Modules
fbStationOne(Enable := NOT testMode,
              ioStation := emConveyor.cm01);
fbStationTwo(Enable := NOT testMode,
              ioStation := emConveyor.cm02);
fbStationThree(Enable := NOT testMode,
                ioStation := emConveyor.cm03);

fbOutStation(Enable := NOT testMode,
              ioStation := emConveyor.cmOut);

fbDrive(Enable := NOT testMode,
         ioDrive := emConveyor.cmDrive,
         stationOne := fbStationOne,
         stationTwo := fbStationTwo,
         stationThree := fbStationThree,
         stationOut := fbOutStation);

// Manage outputs
FC_SetOutput(ioHwInterface := GVL_Abox.uaAboxInterface,
             ioPhysicalControl := emConveyor);
```